

Assignment 2

- 乘法器

请针对理论课件中设计的两种基于8位加法器的乘法器设计进行仿真 (combinational design & repetitive-addition design)。

请从器件利用率、延迟等方面比较不同设计的性能。

代码

Combination design 1

```
library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
entity mult5 is
port (a, b : in std_logic_vector(4 downto 0);
y: out std_logic_vector(9 downto 0));
end entity mult5;
architecture comb1_arch of mult5 is
constant WIDTH : integer := 5;
signal au, bv0, bv1, bv2, bv3, bv4: std_logic_vector(WIDTH-1 downto 0);
signal p0, p1, p2, p3, p4, prod: std_logic_vector(2*WIDTH-1 downto 0);
begin
    au <= a;
    bv0 <= (others => b(0));
    bv1 <= (others => b(1));
    bv2 <= (others => b(2));
    bv3 <= (others => b(3));
    bv4 <= (others => b(4));
    p0 <= "00000" & (bv0 and au);
    p1 <= "0000" & (bv1 and au) & '0';
    p2 <= "000" & (bv2 and au) & "00";
    p3 <= "00" & (bv3 and au) & "000";
    p4 <= '0' & (bv4 and au) & "0000";
    prod <= ((p0+p1)+(p2+p3))+p4;
    y <= prod;
end architecture comb1_arch;
```

Combination design 1 - testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity adder_based_multiplier_tb is
```

```

-- Port ( );
end adder_based_multiplier_tb;

architecture tb of adder_based_multiplier_tb is
component mult5 is
port (a, b : in std_logic_vector(4 downto 0);
      y: out std_logic_vector(9 downto 0));
end component;

signal al_in, bl_in : std_logic_vector(4 downto 0);
signal yl_out : std_logic_vector(9 downto 0);

begin
    ins1: mult5
        port map(
            a => al_in,
            b => bl_in,
            y => yl_out);
    al_in <= "01001" after 10ns, "11100" after 40ns, "01010" after 70ns;
    bl_in <= "10010" after 20ns, "01011" after 50ns, "10110" after 80ns;
end tb;

```

Combination design 2

```

library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
entity mult5 is
port (a, b : in std_logic_vector(4 downto 0);
      y: out std_logic_vector(9 downto 0));
end entity mult5;
architecture comb2_arch of mult5 is
constant WIDTH : integer := 5;
signal au, bv0, bv1, bv2, bv3, bv4: std_logic_vector (WIDTH-1 downto 0);
signal pp0, pp1, pp2, pp3, pp4: std_logic_vector (WIDTH downto 0);
signal prod: std_logic_vector (2*WIDTH-1 downto 0);
begin
    au <= a;
    bv0 <= (others => b(0));
    bv1 <= (others => b(1));
    bv2 <= (others => b(2));
    bv3 <= (others => b(3));
    bv4 <= (others => b(4));

```

```

    pp0 <= '0' & (bv0 and au);
    pp1 <= ('0' & pp0(WIDTH downto 1))+ ('0' & (bv1 and au));
    pp2 <= ('0' & pp1(WIDTH downto 1))+ ('0' & (bv2 and au));
    pp3 <= ('0' & pp2(WIDTH downto 1))+ ('0' & (bv3 and au));
    pp4 <= ('0' & pp3(WIDTH downto 1))+ ('0' & (bv4 and au));
    prod <= pp4 & pp3(0) & pp2(0) & pp1(0) & pp0(0);
    y <= prod;
end architecture comb2_arch;

```

Combination design 2 - testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity adder_based_multiplier_tb is
-- Port ( );
end adder_based_multiplier_tb;

architecture tb of adder_based_multiplier_tb is
component mult5 is
port (a, b : in std_logic_vector(4 downto 0);
      y: out std_logic_vector(9 downto 0));
end component;

signal a1_in, b1_in : std_logic_vector(4 downto 0);
signal y1_out : std_logic_vector(9 downto 0);

begin
    ins1: mult5
        port map(
            a => a1_in,
            b => b1_in,
            y => y1_out);
    a1_in <= "01001" after 10ns, "11100" after 40ns, "01010" after 70ns;
    b1_in <= "10010" after 20ns, "01011" after 50ns, "10110" after 80ns;
end tb;

```

Repetitive-addition design

```

library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
entity seq_mult is

```

```

port (CLK, RESET, start: in std_logic;
a_in, b_in: in std_logic_vector(7 downto 0);
ready: out std_logic;
r: out std_logic_vector(15 downto 0));
end entity seq_mult;
architecture seg_arch of seq_mult is
constant WIDTH : integer :=8;
type state_type is (idle, ab0, load, op);
signal state_reg, state_next : state_type;
signal a_reg, a_next, n_reg, n_next : std_logic_vector (WIDTH-1 downto 0);
signal r_reg, r_next : std_logic_vector (2*WIDTH-1 downto 0);
begin
-- state and data registers
process (CLK, RESET) is
begin
    if RESET = '1' then
        state_reg <= idle;
        a_reg <= "00000000";
        n_reg <="00000000";
        r_reg <= x"0000";
    elsif rising_edge(CLK) then
        state_reg <= state_next;
        a_reg <= a_next;
        n_reg <= n_next;
        r_reg <= r_next;
    end if;
end process;
-- combinational circuit
process (start, state_reg, a_reg, n_reg, r_reg, a_in, b_in) is
begin
-- default value
a_next <= a_reg;
n_next <= n_reg;
r_next <= r_reg;
ready <= '0';
case state_reg is
when idle =>
    if start = '1' then
        if (a_in = "00000000" or b_in = "00000000") then
            state_next <= ab0;
        else
            state_next <= load;
        end if;
    end if;
end process;

```

```

        else
            state_next <= idle;
        end if;
        ready <= '1';
    when ab0 =>
        a_next <= a_in;
        n_next <= b_in;
        r_next <= x"0000";
        state_next <= idle;
    when load =>
        a_next <= a_in;
        n_next <= b_in;
        r_next <= x"0000";
        state_next <= op;
    when op =>
        n_next <= n_reg - 1;
        r_next <= ("00000000" & a_reg) + r_reg;
        if (n_reg = "00000001" ) then
            state_next <= idle;
        else
            state_next <= op;
        end if;
    end case;
end process;
r <= r_reg;
end architecture seg_arch;

```

Repetitive-addition design - testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity seq_mult_tb is
    -- Port ( );
end seq_mult_tb;

architecture tb of seq_mult_tb is
    component seq_mult is
    port (CLK, RESET, start: in std_logic;
        a_in, b_in: in std_logic_vector(7 downto 0);
        ready: out std_logic;
        r: out std_logic_vector(15 downto 0));
    end component;
    signal clk, reset, start, ready : std_logic := '0';

```

```

signal a_in, b_in: std_logic_vector(7 downto 0);
signal r : std_logic_vector(15 downto 0);
begin
    ins: seq_mult
        port map( CLK => clk,
                  RESET => reset,
                  start => start,
                  a_in => a_in,
                  b_in => b_in,
                  ready => ready,
                  r => r);

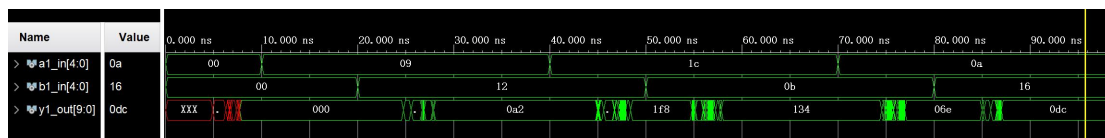
    clk <= not clk after 5ns;
    reset <= '0', '1' after 10ns, '0' after 20ns;
    a_in <= "00000000", "00000011" after 30ns;
    b_in <= "00000000", "00110111" after 30ns;
    start <= '0', '1' after 40ns, '0' after 50ns;

end tb;

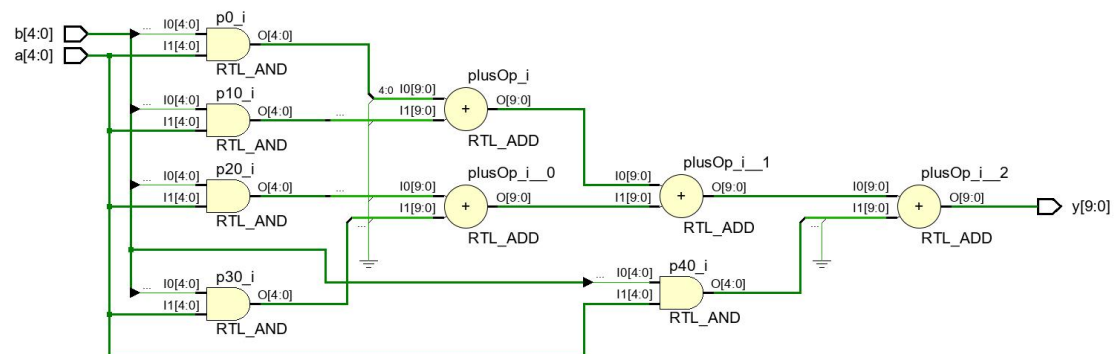
```

结果

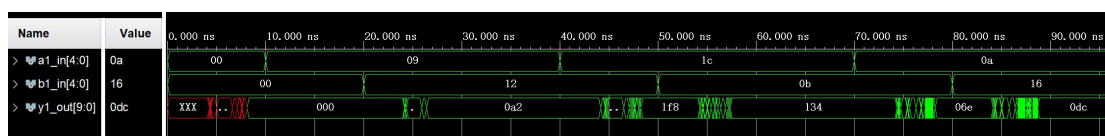
Combination design 1 (post-implementation simulation - Timing)



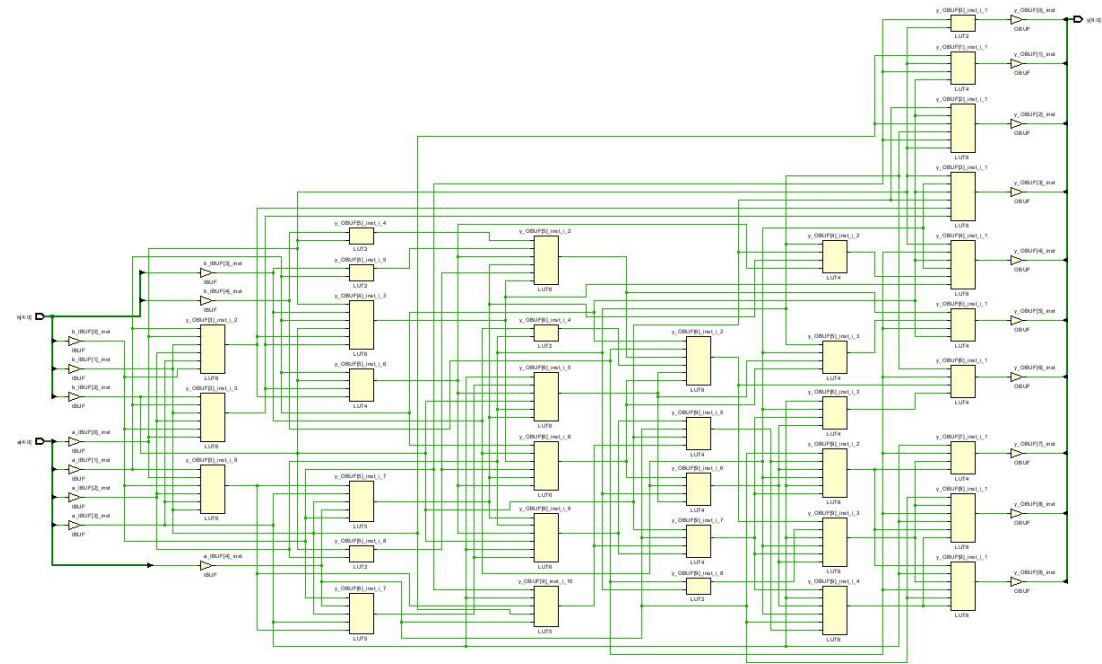
Combination design 1 (RTL schematic)



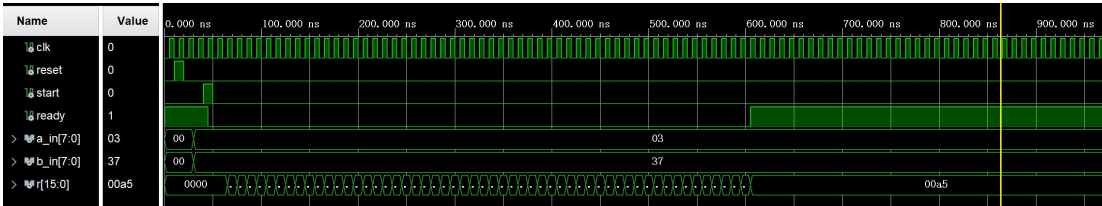
Combination design 2 (post-implementation simulation - Timing)



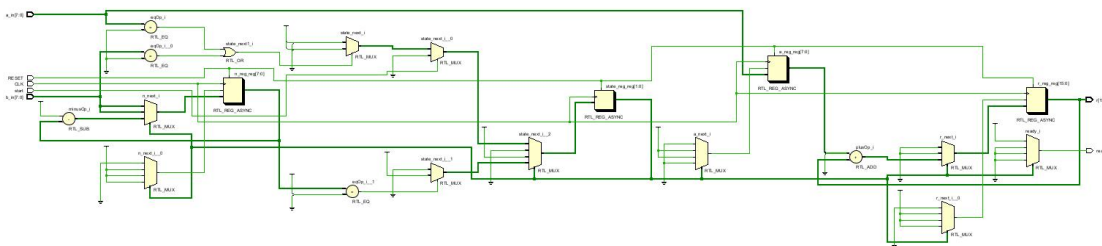
Combination design 2 (RTL schematic)



Repetitive-addition design(post-implementation simulation-Functional)



Repetitive-addition design (RTL schematic)



分析

Combination Design 和 Repetitive-addition Design 的性能比较

1. 器件利用率

Combination Design

器件利用率高: Combination design 1 和 Combination design 2 都是基于组合逻辑的乘法器设计，它们使用了多个加法器来并行计算部分积并求和。这种设计方式使得器件资源的利用率较高，因为乘法器的所有部分都在同时工作。

Repetitive-addition Design

器件利用率低：Repetitive-addition design 是基于时序逻辑的乘法器设计，它使用重复加法的方式进行乘法运算。这种设计方式在每个时钟周期内只执行一次加法操作，器件利用率相对较低。

2. 延迟

Combination Design

延迟小：Combination design 乘法器的运算速度较快，延迟主要由加法和部分积计算的延迟决定。例如，Combination design 1 的延迟主要来自 5 个部分积的计算和它们的求和过程，但由于是并行计算，所以总延迟相对较小。

Repetitive-addition Design

延迟大：Repetitive-addition Design 乘法器的运算速度较慢，因为需要多个时钟周期来完成乘法运算。延迟由输入数的大小决定，每个时钟周期内执行一次加法操作和移位操作，因此总延迟相对较大。

3. 消耗的硬件资源

Combination Design

资源消耗大：由于需要同时处理所有的位运算和加法操作，组合逻辑乘法器会消耗较多的逻辑门和硬件资源。例如，在 Combination design 1 中，有 5 个部分积计算单元和一个 10 位的加法器来求和，导致器件资源占用较多。

Repetitive-addition Design

资源消耗小：与组合逻辑乘法器相比，重复加法乘法器的资源消耗较小。它只需要一个加法器和少量的寄存器来存储中间结果和计数器，因此在器件资源占用方面较为节省。

Assignment 3

- 乘法器

参考理论课件中 pipelined design 8 位乘法器，设计一个 n 位的乘法器流水线设计，其中 n 是 1 到 9 之间的正整数。对该设计进行仿真，指明延迟和吞吐量，并将其与 Assignment 2 中的设计进行比较。

代码

Pipelined multiplier

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity Pipelined_multiplier is
  Port (clk : in std_logic;
        reset : in std_logic;
        b : in std_logic_vector(8 downto 0);
```



```

        a : in std_logic_vector(8 downto 0);
        n : in std_logic_vector(3 downto 0);
        y : out std_logic_vector(17 downto 0) );
end Pipelined_multiplier;

```

```

architecture pipe_arch of Pipelined_multiplier is
constant WIDTH : integer := 9;
signal a2_reg, a3_reg, a4_reg, a5_reg, a6_reg, a7_reg, a8_reg :
std_logic_vector (WIDTH-1 downto 0);
signal b2_reg, b3_reg, b4_reg, b5_reg, b6_reg, b7_reg, b8_reg :
std_logic_vector (WIDTH-1 downto 0);
signal a1, a2_next, a3_next, a4_next, a5_next, a6_next, a7_next,
a8_next:std_logic_vector (WIDTH-1 downto 0);
signal b1, b2_next, b3_next, b4_next, b5_next, b6_next, b7_next,
b8_next:std_logic_vector (WIDTH-1 downto 0);
signal bv0, bv1, bv2, bv3, bv4, bv5, bv6, bv7, bv8: std_logic_vector
(WIDTH-1 downto 0);
signal bp0, bp1, bp2, bp3, bp4, bp5, bp6, bp7, bp8: std_logic_vector
(2*WIDTH-1 downto 0);
signal pp1_reg, pp2_reg, pp3_reg, pp4_reg, pp5_reg, pp6_reg, pp7_reg,
pp8_reg:std_logic_vector (2*WIDTH-1 downto 0);
signal pp0, pp1_next, pp2_next, pp3_next, pp4_next, pp5_next, pp6_next,
pp7_next, pp8_next: std_logic_vector (2*WIDTH-1 downto 0);
begin
    -- pipeline registers
    process (clk, reset)
    begin
        if (reset = '1') then
            pp1_reg <= (others => '0');
            pp2_reg <= (others => '0');
            pp3_reg <= (others => '0');
            pp4_reg <= (others => '0');
            pp5_reg <= (others => '0');
            pp6_reg <= (others => '0');
            pp7_reg <= (others => '0');
            pp8_reg <= (others => '0');

            a2_reg <= (others => '0');
            a3_reg <= (others => '0');
            a4_reg <= (others => '0');
            a5_reg <= (others => '0');
            a6_reg <= (others => '0');
            a7_reg <= (others => '0');
            a8_reg <= (others => '0');

```

```

        b2_reg <= (others => '0');
        b3_reg <= (others => '0');
        b4_reg <= (others => '0');
        b5_reg <= (others => '0');
        b6_reg <= (others => '0');
        b7_reg <= (others => '0');
        b8_reg <= (others => '0');
    elsif (clk'event and clk = '1') then
        pp1_reg <= pp1_next;
        pp2_reg <= pp2_next;
        pp3_reg <= pp3_next;
        pp4_reg <= pp4_next;
        pp5_reg <= pp5_next;
        pp6_reg <= pp6_next;
        pp7_reg <= pp7_next;
        pp8_reg <= pp8_next;

        a2_reg <= a2_next;
        a3_reg <= a3_next;
        a4_reg <= a4_next;
        a5_reg <= a5_next;
        a6_reg <= a6_next;
        a7_reg <= a7_next;
        a8_reg <= a8_next;

        b2_reg <= b2_next;
        b3_reg <= b3_next;
        b4_reg <= b4_next;
        b5_reg <= b5_next;
        b6_reg <= b6_next;
        b7_reg <= b7_next;
        b8_reg <= b8_next;
    end if;
end process;

-- merged stage 0 & 1 for pipeline
bv0 <= (others => b(0));
bp0 <= "000000000" & (bv0 and a);
pp0 <= bp0;
a1 <= a;
b1 <= b;
--
bv1 <= (others => b1(1));

```

```

bp1 <= "00000000" & (bv1 and a1) & "0";
pp1_next <= pp0 + bp1;
a2_next <= a1;
b2_next <= b1;

-- stage 2
bv2 <= (others => b2_reg(2));
bp2 <= "0000000" & (bv2 and a2_reg) & "00";
pp2_next <= pp1_reg + bp2;
a3_next <= a2_reg;
b3_next <= b2_reg;

-- stage 3
bv3 <= (others => b3_reg(3));
bp3 <= "000000" & (bv3 and a3_reg) & "000";
pp3_next <= pp2_reg + bp3;
a4_next <= a3_reg;
b4_next <= b3_reg;

-- stage 4
bv4 <= (others => b4_reg(4));
bp4 <= "00000" & (bv4 and a4_reg) & "0000";
pp4_next <= pp3_reg + bp4;
a5_next <= a4_reg;
b5_next <= b4_reg;

-- stage 5
bv5 <= (others => b5_reg(5));
bp5 <= "0000" & (bv5 and a5_reg) & "00000";
pp5_next <= pp4_reg + bp5;
a6_next <= a5_reg;
b6_next <= b5_reg;

-- stage 6
bv6 <= (others => b6_reg(6));
bp6 <= "000" & (bv6 and a6_reg) & "000000";
pp6_next <= pp5_reg + bp6;
a7_next <= a6_reg;
b7_next <= b6_reg;

-- stage 7
bv7 <= (others => b7_reg(7));
bp7 <= "00" & (bv7 and a7_reg) & "00000000";
pp7_next <= pp6_reg + bp7;

```

```

a8_next <= a7_reg;
b8_next <= b7_reg;

-- stage 8
bv8 <= (others => b8_reg(8));
bp8 <= "0" & (bv8 and a8_reg) & "00000000";
pp8_next <= pp7_reg + bp8;

-- output
process(n, pp0, pp1_reg, pp2_reg, pp3_reg, pp4_reg, pp5_reg, pp6_reg,
pp7_reg, pp8_reg)
begin
if n = "0001" then
y <= pp0;
elsif n = "0010" then
y <= pp1_reg;
elsif n = "0011" then
y <= pp2_reg;
elsif n = "0100" then
y <= pp3_reg;
elsif n = "0101" then
y <= pp4_reg;
elsif n = "0110" then
y <= pp5_reg;
elsif n = "0111" then
y <= pp6_reg;
elsif n = "1000" then
y <= pp7_reg;
elsif n = "1001" then
y <= pp8_reg;
end if;
end process;
end architecture pipe_arch;

```

Pipelined multiplier - testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Pipelined_multiplier_tb is
-- Port ( );
end Pipelined_multiplier_tb;

architecture tb of Pipelined_multiplier_tb is

```

```

component Pipelined_multiplier is
  Port (clk : in std_logic;
        reset : in std_logic;
        b : in std_logic_vector(8 downto 0);
        a : in std_logic_vector(8 downto 0);
        n : in std_logic_vector(3 downto 0);
        y : out std_logic_vector(17 downto 0) );
end component;

signal clk, reset : std_logic := '0';
signal b, a : std_logic_vector(8 downto 0);
signal n : std_logic_vector(3 downto 0);
signal y : std_logic_vector(17 downto 0);
begin
  ins1: Pipelined_multiplier
    port map (clk => clk,
              reset => reset,
              b => b,
              a => a,
              n => n,
              y => y);

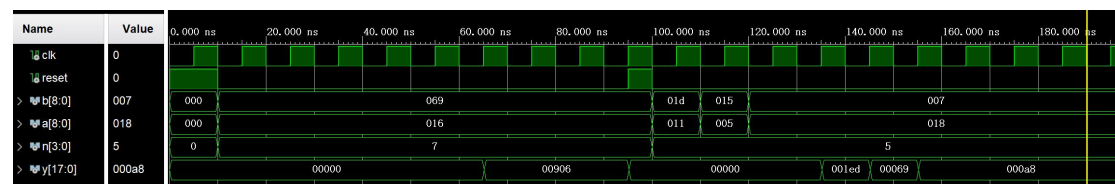
  clk <= not clk after 5ns;
  reset <= '1', '0' after 10ns, '1' after 95ns, '0' after 100ns;
  b <= "000000000", "001101001" after 10ns, "000011101" after 100ns;
  a <= "000000000", "000010110" after 10ns, "000010001" after 100ns;
  n <= "0000", "0111" after 10ns, "0101" after 100ns;

end tb;

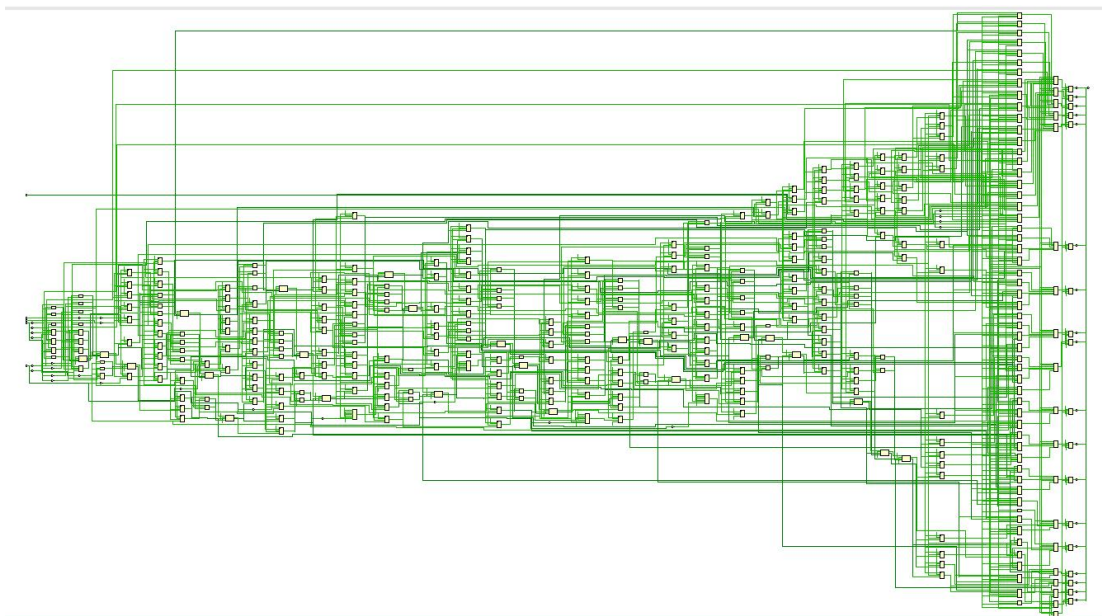
```

结果

Pipelined design(post-implementation simulation-Functional)



Pipelined design (RTL schematic)



分析

Pipelined design 的延迟：由于流水线设计将 n 位的乘法过程分解为多个阶段，因此总延迟由整个流水线的深度（即乘法器的位数）决定。对于 n 位乘法器，流水线有 n 个阶段，完成一次计算任务的总延迟为 $(n - 1)$ 个时钟周期。

Pipelined design 的吞吐量：对于 n 位乘法器，完成一次计算任务的总延迟为 $(n - 1)$ 个时钟周期。由于流水线设计的特性，完成 k 个 n 位计算任务需要的总时长为 $(n + k - 2)$ 个时钟周期 T ，吞吐量为 $k / ((n + k - 2) * T)$ ，当 k 很大时吞吐量近似于 $(1 / T)$ 。

和 Combination design 进行比较：

硬件资源：Combination Design 在硬件资源方面具有较高的逻辑门和加法器资源消耗，但寄存器使用较少，适合对延迟要求极高的应用场景。

Pipelined Design 在硬件资源方面具有适量的加法器和大量的寄存器资源，适合需要高吞吐量的应用场景。虽然总延迟较大，但每个阶段的延迟较小，且能够通过流水线的方式提高运算效率。

和 Repetitive-addition Design 进行比较：

吞吐量比较：Repetitive-addition design 的吞吐量较低，因为它在一个乘法运算完成之前无法开始新的运算。而 Pipelined design 能够在在一个时钟周期内启动一个新的乘法运算，从而具有更高的吞吐量。